Assignment 4 — Java Collections

Name: CHETANYA **Roll No.:** 1024030027 **Group:** 3C35

Question 1: Course Registration Portal

Problem Statement

Design a comprehensive course registration portal combining multiple collection types for optimal data handling, prerequisites tracking, and waitlist management. Use a HashMap<String, Course> for fast course lookups by course code. Each Course object must contain a HashSet of unique enrolled student IDs, an ArrayList for ordered prerequisite codes, and a LinkedList acting as a first-come, first-served waitlist queue. Handle registration requests with capacity checks and automated waitlist promotion logic.

Code — CourseRegistrationPortal.java

```
import java.util.*;

class Course {
    private final String code;
    private final String title;
    private final int capacity;
    private final Set<String> enrolledStudents = new HashSet<>();
    private final List<String> prerequisites = new ArrayList<>();
    private final LinkedList<String> waitlist = new LinkedList<>();

    Course(String code, String title, int capacity) {
        this.code = code;
        this.title = title;
        this.capacity = capacity;
    }

    String getCode() { return code; }
    String getTitle() { return title; }
    int getCapacity() { return capacity; }
    Set<String> getEnrolledStudents() { return enrolledStudents; }
    List<String> getPrerequisites() { return prerequisites; }
    LinkedList<String> getWaitlist() { return waitlist; }

    void addPrerequisite(String courseCode) {
        if (!prerequisites.contains(courseCode)) {
            prerequisites.add(courseCode);
        }
    }

    boolean isFull() { return enrolledStudents.size() >= capacity; }

    @Override
    public String toString() {
        return String.format("%s - %s | Seats: %d/%d | Prereqs: %s | Enrolled: %s | Waitlist: %s",
                code, title, enrolledStudents.size(), capacity, prerequisites, enrolledStudents,
waitlist);
    }
}

public class CourseRegistrationPortal {
    private final Map<String, Course> courses = new HashMap<>();
    private final Map<String, Set<String>> completedCourses = new HashMap<>();

    public void addCourse(Course course) {
        courses.put(course.getCode(), course);
```

```
    }

    public void markCompleted(String studentId, String courseCode) {
        completedCourses.computeIfAbsent(studentId, k -> new HashSet<>()).add(courseCode);
    }

    private List<String> missingPrerequisites(String studentId, Course course) {
        Set<String> done = completedCourses.getOrDefault(studentId, Collections.emptySet());
        List<String> missing = new ArrayList<>();
        for (String prereq : course.getPrerequisites()) {
            if (!done.contains(prereq)) {
                missing.add(prereq);
            }
        }
        return missing;
    }

    public String register(String studentId, String courseCode) {
        Course course = courses.get(courseCode);
        if (course == null) {
            return "FAILED: course " + courseCode + " does not exist";
        }
        if (course.getEnrolledStudents().contains(studentId)) {
            return "FAILED: " + studentId + " is already enrolled in " + courseCode;
        }
        if (course.getWaitlist().contains(studentId)) {
            return "FAILED: " + studentId + " is already on the waitlist for " + courseCode
                    + " (position " + (course.getWaitlist().indexOf(studentId) + 1) + ")";
        }
        List<String> missing = missingPrerequisites(studentId, course);
        if (!missing.isEmpty()) {
            return "FAILED: " + studentId + " is missing prerequisites " + missing + " for " +
courseCode;
        }
        if (course.isFull()) {
            course.getWaitlist().addLast(studentId);
            return "WAITLISTED: " + studentId + " for " + courseCode
                    + " (position " + course.getWaitlist().size() + ")";
        }
        course.getEnrolledStudents().add(studentId);
        return "ENROLLED: " + studentId + " in " + courseCode;
    }

    public String drop(String studentId, String courseCode) {
        Course course = courses.get(courseCode);
        if (course == null) {
            return "FAILED: course " + courseCode + " does not exist";
        }
        if (course.getWaitlist().remove(studentId)) {
            return "REMOVED: " + studentId + " from the waitlist of " + courseCode;
        }
        if (!course.getEnrolledStudents().remove(studentId)) {
            return "FAILED: " + studentId + " is not registered for " + courseCode;
        }
        return "DROPPED: " + studentId + " from " + courseCode + promoteFromWaitlist(course);
    }

    private String promoteFromWaitlist(Course course) {
        if (course.isFull() || course.getWaitlist().isEmpty()) {
            return "";
        }
        String promoted = course.getWaitlist().pollFirst();
        course.getEnrolledStudents().add(promoted);
        return " -> PROMOTED " + promoted + " from the waitlist";
    }

    public void printCourse(String courseCode) {
        Course course = courses.get(courseCode);
        System.out.println(course == null ? "No course " + courseCode : course);
    }
```

```
    public static void main(String[] args) {
        CourseRegistrationPortal portal = new CourseRegistrationPortal();

        Course cs101 = new Course("CS101", "Programming Fundamentals", 3);
        Course cs201 = new Course("CS201", "Data Structures", 2);
        cs201.addPrerequisite("CS101");
        Course cs301 = new Course("CS301", "Algorithms", 2);
        cs301.addPrerequisite("CS101");
        cs301.addPrerequisite("CS201");

        portal.addCourse(cs101);
        portal.addCourse(cs201);
        portal.addCourse(cs301);

        for (String s : new String[]{"S1", "S2", "S3", "S4"}) {
            portal.markCompleted(s, "CS101");
        }

        System.out.println("=== Registration requests ===");
        System.out.println(portal.register("S1", "CS201"));
        System.out.println(portal.register("S2", "CS201"));
        System.out.println(portal.register("S3", "CS201"));
        System.out.println(portal.register("S4", "CS201"));
        System.out.println(portal.register("S5", "CS201"));
        System.out.println(portal.register("S1", "CS201"));
        System.out.println(portal.register("S3", "CS201"));
        System.out.println(portal.register("S1", "CS301"));
        System.out.println(portal.register("S1", "CS999"));
        System.out.println(portal.register("S5", "CS101"));
        portal.printCourse("CS201");

        System.out.println("\n=== Drops and waitlist promotion ===");
        System.out.println(portal.drop("S1", "CS201"));
        portal.printCourse("CS201");
        System.out.println(portal.drop("S4", "CS201"));
        System.out.println(portal.drop("S2", "CS201"));
        System.out.println(portal.drop("S9", "CS201"));
        portal.printCourse("CS201");

        System.out.println("\n=== All courses ===");
        for (Course c : portal.courses.values()) {
            System.out.println(c);
        }
    }
}
```

Output

```
(base) chetanyabhargav@MacBook-Air-13 mini-redis %  /usr/bin/env /Us
ers/chetanyabhargav/.vscode/extens
ions/redhat.java-1.57.0-darwin-arm
64/jre/25.0.4.1-macosx-aarch64/bin
/java -XX:+ShowCodeDetailsInExcept
ionMessages -cp /Users/chetanyabha
rgav/Library/Application\ Support/
Code/User/workspaceStorage/c3d380d
3ed601a077a76ad76a9512148/redhat.j
ava/jdt_ws/jdt.ls-java-project/bin
 CourseRegistrationPortal
=== Registration requests ===
ENROLLED: S1 in CS201
ENROLLED: S2 in CS201
WAITLISTED: S3 for CS201 (position 1)
WAITLISTED: S4 for CS201 (position 2)
FAILED: S5 is missing prerequisites [CS101] for CS201
FAILED: S1 is already enrolled in CS201
FAILED: S3 is already on the waitlist for CS201 (position 1)
FAILED: S1 is missing prerequisites [CS201] for CS301
FAILED: course CS999 does not exist
ENROLLED: S5 in CS101
CS201 - Data Structures | Seats: 2/2 | Prereqs: [CS101] | Enrolled: [S1, S2] | Waitlist: [S3, S4]

=== Drops and waitlist promotion ===
DROPPED: S1 from CS201 -> PROMOTED S3 from the waitlist
CS201 - Data Structures | Seats: 2/2 | Prereqs: [CS101] | Enrolled: [S3, S2] | Waitlist: [S4]
REMOVED: S4 from the waitlist of CS201
DROPPED: S2 from CS201
FAILED: S9 is not registered for CS201
CS201 - Data Structures | Seats: 1/2 | Prereqs: [CS101] | Enrolled: [S3] | Waitlist: []

=== All courses ===
CS301 - Algorithms | Seats: 0/2 | Prereqs: [CS101, CS201] | Enrolled: [] | Waitlist: []
CS101 - Programming Fundamentals | Seats: 1/3 | Prereqs: [] | Enrolled: [S5] | Waitlist: []
CS201 - Data Structures | Seats: 1/2 | Prereqs: [CS101] | Enrolled: [S3] | Waitlist: []
(base) chetanyabhargav@MacBook-Air-13 mini-redis %
```

Question 2: Media Player Queue Engine

Problem Statement

Build a media player queue engine that supports fast insertions at arbitrary positions, recent history tracking, and bidirectional navigation. Utilize a LinkedList<Song> to manage playback queues. Implement playlist features that allow adding songs to the front (priority queueing), removing duplicate song entries efficiently, and using ListIterator to traverse the playlist both forward and backward without index-based performance bottlenecks.

Code — MediaPlayerQueue.java

```
import java.util.*;

class Song {
    private final String title;
    private final String artist;
    private final int durationSeconds;

    Song(String title, String artist, int durationSeconds) {
        this.title = title;
        this.artist = artist;
        this.durationSeconds = durationSeconds;
    }

    String getTitle() { return title; }
    String getArtist() { return artist; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Song)) return false;
        Song other = (Song) o;
        return title.equalsIgnoreCase(other.title) && artist.equalsIgnoreCase(other.artist);
    }

    @Override
    public int hashCode() {
        return Objects.hash(title.toLowerCase(), artist.toLowerCase());
    }

    @Override
    public String toString() {
        return String.format("%s - %s (%d:%02d)", title, artist, durationSeconds / 60, durationSeconds
% 60);
    }
}

public class MediaPlayerQueue {
    private static final int HISTORY_LIMIT = 5;

    private final LinkedList<Song> queue = new LinkedList<>();
    private final LinkedList<Song> history = new LinkedList<>();

    private ListIterator<Song> cursor = queue.listIterator();
    private Song current;
    private boolean movedForward = true;

    private int currentIndex() {
        if (current == null) return -1;
        return movedForward ? cursor.previousIndex() : cursor.nextIndex();
    }

    private void resetCursor(int newCurrentIndex) {
        if (newCurrentIndex < 0) {
```

```
            current = null;
            cursor = queue.listIterator();
        } else {
            cursor = queue.listIterator(newCurrentIndex + 1);
        }
        movedForward = true;
    }


    public void addToEnd(Song song) {
        int idx = currentIndex();
        queue.addLast(song);
        resetCursor(idx);
        System.out.println("Added to end      : " + song);
    }

    public void addToFront(Song song) {
        int idx = currentIndex();
        queue.addFirst(song);
        resetCursor(idx < 0 ? -1 : idx + 1);
        System.out.println("Added to front    : " + song);
    }

    public void playNextInQueue(Song song) {
        if (current == null) {
            addToFront(song);
            return;
        }
        if (!movedForward) {
            cursor.next();
            movedForward = true;
        }
        cursor.add(song);
        cursor.previous();
        System.out.println("Play next         : " + song);
    }

    public void insertAt(int position, Song song) {
        if (position < 0 || position > queue.size()) {
            System.out.println("Invalid position " + position + " for " + song);
            return;
        }
        int idx = currentIndex();
        ListIterator<Song> it = queue.listIterator(position);
        it.add(song);
        resetCursor(idx >= 0 && position <= idx ? idx + 1 : idx);
        System.out.println("Inserted at " + position + "     : " + song);
    }

    public void removeDuplicates() {
        int idx = currentIndex();
        int newIdx = -1;
        Map<Song, Integer> firstPosition = new HashMap<>();
        Set<Song> seen = new HashSet<>();
        int removed = 0;
        int kept = 0;

        ListIterator<Song> it = queue.listIterator();
        while (it.hasNext()) {
            int oldIndex = it.nextIndex() + removed;
            Song song = it.next();
            if (!seen.add(song)) {
                it.remove();
                removed++;
                if (oldIndex == idx) newIdx = firstPosition.get(song);
            } else {
                firstPosition.put(song, kept);
                if (oldIndex == idx) newIdx = kept;
                kept++;
            }
        }
```

```
        if (newIdx >= 0) current = queue.get(newIdx);
        resetCursor(newIdx);
        System.out.println("Removed " + removed + " duplicate entr" + (removed == 1 ? "y" : "ies"));
    }


    public Song next() {
        if (!movedForward && cursor.hasNext()) {
            cursor.next();
        }
        movedForward = true;
        if (!cursor.hasNext()) {
            System.out.println("End of queue");
            return null;
        }
        current = cursor.next();
        play(current, "Next     ");
        return current;
    }

    public Song previous() {
        if (movedForward && current != null && cursor.hasPrevious()) {
            cursor.previous();
        }
        movedForward = false;
        if (!cursor.hasPrevious()) {
            System.out.println("Start of queue");
            if (current != null) {
                cursor.next();
                movedForward = true;
            }
            return null;
        }
        current = cursor.previous();
        play(current, "Previous ");
        return current;
    }

    private void play(Song song, String action) {
        System.out.println(action + "> Now playing: " + song);
        history.remove(song);
        history.addFirst(song);
        if (history.size() > HISTORY_LIMIT) {
            history.removeLast();
        }
    }


    public void showQueue() {
        int idx = currentIndex();
        System.out.println("---- Queue (" + queue.size() + " songs) ----");
        int i = 0;
        for (Song s : queue) {
            System.out.println((i == idx ? " >> " : "    ") + (i + 1) + ". " + s);
            i++;
        }
    }

    public void showHistory() {
        System.out.println("---- Recently played (latest first) ----");
        for (Song s : history) {
            System.out.println("    " + s);
        }
    }

    public static void main(String[] args) {
        MediaPlayerQueue player = new MediaPlayerQueue();

        Song s1 = new Song("Blinding Lights", "The Weeknd", 200);
        Song s2 = new Song("Shape of You", "Ed Sheeran", 233);
        Song s3 = new Song("Believer", "Imagine Dragons", 204);
```

```
        Song s4 = new Song("Perfect", "Ed Sheeran", 263);
        Song s5 = new Song("Levitating", "Dua Lipa", 203);
        Song s6 = new Song("Kesariya", "Arijit Singh", 268);

        System.out.println("=== Building the queue ===");
        player.addToEnd(s1);
        player.addToEnd(s2);
        player.addToEnd(s3);
        player.addToEnd(new Song("shape of you", "ed sheeran", 233));
        player.addToEnd(s4);
        player.addToFront(s5);
        player.insertAt(3, s6);
        player.addToEnd(s1);
        player.showQueue();

        System.out.println("\n=== Forward navigation ===");
        player.next();
        player.next();
        player.next();
        player.showQueue();

        System.out.println("\n=== Backward navigation ===");
        player.previous();
        player.previous();
        player.previous();
        player.next();

        System.out.println("\n=== Removing duplicates ===");
        player.removeDuplicates();
        player.showQueue();

        System.out.println("\n=== Play next + priority while playing ===");
        player.playNextInQueue(new Song("Heat Waves", "Glass Animals", 238));
        player.addToFront(new Song("Tum Hi Ho", "Arijit Singh", 262));
        player.showQueue();
        player.next();
        player.next();

        System.out.println("\n=== Play to the end ===");
        while (player.next() != null) { }
        player.previous();

        System.out.println();
        player.showHistory();
    }
}
```

Output

```
(base) chetanyabhargav@MacBook-Air-13 mini-redis %  /usr/bin/env /Users/chetanyabhargav/.vscode/
extensions/redhat.java-1.57.0-darwin-arm64/jre/25.0.4.1-macosx-aarch64/bin/java -XX:+ShowCodeDet
ailsInExceptionMessages -cp /Users/chetanyabhargav/Library/Application\ Support/Code/User/worksp
aceStorage/c3d380d3ed601a077a76ad76a9512148/redhat.java/jdt_ws/jdt.ls-java-project/bin MediaPlay
erQueue
=== Building the queue ===
Added to end      : Blinding Lights - The Weeknd (3:20)
Added to end      : Shape of You - Ed Sheeran (3:53)
Added to end      : Believer - Imagine Dragons (3:24)
Added to end      : shape of you - ed sheeran (3:53)
Added to end      : Perfect - Ed Sheeran (4:23)
Added to front    : Levitating - Dua Lipa (3:23)
Inserted at 3     : Kesariya - Arijit Singh (4:28)
Added to end      : Blinding Lights - The Weeknd (3:20)
---- Queue (8 songs) ----
    1. Levitating - Dua Lipa (3:23)
    2. Blinding Lights - The Weeknd (3:20)
    3. Shape of You - Ed Sheeran (3:53)
    4. Kesariya - Arijit Singh (4:28)
    5. Believer - Imagine Dragons (3:24)
    6. shape of you - ed sheeran (3:53)
    7. Perfect - Ed Sheeran (4:23)
    8. Blinding Lights - The Weeknd (3:20)

=== Forward navigation ===
Next     > Now playing: Levitating - Dua Lipa (3:23)
Next     > Now playing: Blinding Lights - The Weeknd (3:20)
Next     > Now playing: Shape of You - Ed Sheeran (3:53)
---- Queue (8 songs) ----
    1. Levitating - Dua Lipa (3:23)
    2. Blinding Lights - The Weeknd (3:20)
 >> 3. Shape of You - Ed Sheeran (3:53)
    4. Kesariya - Arijit Singh (4:28)
    5. Believer - Imagine Dragons (3:24)
    6. shape of you - ed sheeran (3:53)
    7. Perfect - Ed Sheeran (4:23)
    8. Blinding Lights - The Weeknd (3:20)

=== Backward navigation ===
Previous > Now playing: Blinding Lights - The Weeknd (3:20)
Previous > Now playing: Levitating - Dua Lipa (3:23)
Start of queue
Next     > Now playing: Blinding Lights - The Weeknd (3:20)

=== Removing duplicates ===
Removed 2 duplicate entries
---- Queue (6 songs) ----
    1. Levitating - Dua Lipa (3:23)
 >> 2. Blinding Lights - The Weeknd (3:20)
    3. Shape of You - Ed Sheeran (3:53)
    4. Kesariya - Arijit Singh (4:28)
    5. Believer - Imagine Dragons (3:24)
    6. Perfect - Ed Sheeran (4:23)
```

Output (continued)

```
=== Removing duplicates ===
Removed 2 duplicate entries
---- Queue (6 songs) ----
    1. Levitating - Dua Lipa (3:23)
 >> 2. Blinding Lights - The Weeknd (3:20)
    3. Shape of You - Ed Sheeran (3:53)
    4. Kesariya - Arijit Singh (4:28)
    5. Believer - Imagine Dragons (3:24)
    6. Perfect - Ed Sheeran (4:23)

=== Play next + priority while playing ===
Play next         : Heat Waves - Glass Animals (3:58)
Added to front    : Tum Hi Ho - Arijit Singh (4:22)
---- Queue (8 songs) ----
    1. Tum Hi Ho - Arijit Singh (4:22)
    2. Levitating - Dua Lipa (3:23)
 >> 3. Blinding Lights - The Weeknd (3:20)
    4. Heat Waves - Glass Animals (3:58)
    5. Shape of You - Ed Sheeran (3:53)
    6. Kesariya - Arijit Singh (4:28)
    7. Believer - Imagine Dragons (3:24)
    8. Perfect - Ed Sheeran (4:23)
Next     > Now playing: Heat Waves - Glass Animals (3:58)
Next     > Now playing: Shape of You - Ed Sheeran (3:53)

=== Play to the end ===
Next     > Now playing: Kesariya - Arijit Singh (4:28)
Next     > Now playing: Believer - Imagine Dragons (3:24)
Next     > Now playing: Perfect - Ed Sheeran (4:23)
End of queue
Previous > Now playing: Believer - Imagine Dragons (3:24)

---- Recently played (latest first) ----
    Believer - Imagine Dragons (3:24)
    Perfect - Ed Sheeran (4:23)
    Kesariya - Arijit Singh (4:28)
    Shape of You - Ed Sheeran (3:53)
    Heat Waves - Glass Animals (3:58)
(base) chetanyabhargav@MacBook-Air-13 mini-redis %
```